

Ghulam Ishaq Khan Institute of Engineering Sciences and Technology (GIKI)

ASSIGNMENT #1 — Project Report Component 1

Subject Database Management System

Course CS-232-A-Spring-26

Class BSAI — 4th Semester, Batch 34 (Fall-24)

Instructor M. Qasim Riaz

1. Project Title

Hospify — Hospital Management System

2. Group Members

- | | | |
|----|-----------------------|---------|
| 1. | Atta Ur Rahman Sheikh | 2024122 |
| 2. | Ibrahim Umar | 2024413 |
| 3. | Talal Iqbal | 2024476 |

3. Project Description

Hospify is a role-based Hospital Management System that unifies patient records, appointments, admissions, beds, pharmacy, lab orders, and billing into a single secure workspace. It pairs a **React + Vite** frontend with a **Flask + PostgreSQL** backend, layered with JWT authentication and a full audit trail. Real-time bed status updates are powered by **Firestore (Firebase)**, ensuring ward occupancy is always current across all connected clients.

4. Types of Users

Hospify defines **seven distinct roles**, each enforced by route guards in the backend and sidebar gates in the frontend:

 tabheadblue

1	Super Admin	Manage staff accounts, view audit log, resolve alerts, push/pull cloud backups, full access to every module.
2	Doctor	View daily schedule, consult patients, write prescriptions, order lab tests, complete appointments.
3	Nurse	Manage wards & admissions, record patient vitals, raise critical alerts, update bed status.
4	Receptionist	Register new patients, schedule appointments, admit patients and assign beds.
5	Pharmacist	View prescriptions, dispense medicines, manage inventory and restock.
6	Lab Technician	Process lab orders and upload results.
7	Billing Staff	Generate bills from admissions and record payments.

5. User-Specific Detailed Requirements

5.1 Super Admin

- Create, edit, activate, or deactivate staff accounts across all roles.
- View the complete system audit log (all table changes with old/new snapshots).
- Resolve active alerts (low-stock, abnormal vitals).
- Push and pull cloud backups to/from Firebase.
- Access every module with full read/write permissions.

5.2 Doctor

- View today's appointment schedule with status counts.
- Open and consult a patient's admission record.
- Write prescriptions linked to an active admission.
- Order lab tests for admitted patients and set priority.
- Mark appointments as completed or cancelled.

5.3 Nurse

- View and manage ward admissions.
- Record patient vitals (temperature, blood pressure, pulse, SpO₂, weight).
- View automatically raised abnormal-vitals alerts.
- Update bed status in real time via Firestore.

5.4 Receptionist

- Register new patients with full demographic details (CNIC, blood group, emergency contact).
- Search and view existing patient records.
- Schedule, reschedule, or cancel appointments.
- Admit patients and assign them to available beds.

5.5 Pharmacist

- a. View pending prescriptions from doctors.
- b. Dispense medicines against prescription line items.
- c. View current medicine inventory and stock levels.
- d. Restock medicines and update inventory quantities.
- e. View low-stock alerts for medicines below reorder level.

5.6 Lab Technician

- a. View the queue of pending and in-progress lab orders.
- b. Process a lab order and upload result file.
- c. Add result summary notes to completed orders.

5.7 Billing Staff

- a. Generate itemised bills from admission records (bed days, lab tests, prescriptions).
- b. View bill details with auto-computed line totals.
- c. Record payments and update bill status.
- d. View payment history for an admission.

6. Major Modules

1. **Authentication & User Management** — Login, JWT refresh/logout, staff CRUD, role toggling.
2. **Patient Records** — Patient registration, search, demographic management.
3. **Doctors & Appointments** — Doctor directory, appointment scheduling, schedule view.
4. **Admissions & Bed Management** — Patient admissions, discharge, real-time bed status via Firestore.
5. **Vitals Monitoring** — Nurse-driven vitals recording, average computation, abnormal-vitals detection.
6. **Pharmacy & Prescriptions** — Prescription writing, medicine dispensing, inventory tracking.
7. **Lab Orders & Results** — Lab order creation, technician queue, result upload and storage.
8. **Billing & Payments** — Bill generation, itemised breakdown, payment recording.
9. **Audit, Alerts & Cloud Backup** — Audit log, low-stock alerts, abnormal-vitals alerts, Firebase cloud backup.
10. **Media / File Storage** — Lab result file uploads stored as BYTEA with optional Firebase URL.

7. Technical Details

7.1 Database Tables

All 21 tables are in 3NF and are listed below in FK-safe creation order.

tabheadblue	
users	user_id (PK), full_name, email (UNIQUE), password_hash, role, is_active, created_at
patients	patient_id (PK), full_name, dob, gender, cnic (UNIQUE), phone, address, blood_group, emergency_contact, created_at
wards	ward_id (PK), ward_name, ward_type, floor_number, total_beds
beds	bed_id (PK), ward_id (FK→wards), bed_number, bed_type, status, UNIQUE(ward_id, bed_number)
doctors	doctor_id (PK), user_id (FK→users, UNIQUE), specialization, qualification, phone, available_days
nurses	nurse_id (PK), user_id (FK→users, UNIQUE), ward_id (FK→wards), shift
lab_technicians	tech_id (PK), user_id (FK→users, UNIQUE)
pharmacists	pharmacist_id (PK), user_id (FK→users, UNIQUE)
admissions	admission_id (PK), patient_id (FK), bed_id (FK), doctor_id (FK), admitted_at, discharged_at, admission_type, status, notes
appointments	appointment_id (PK), patient_id (FK), doctor_id (FK), scheduled_at, status, reason
vitals	vital_id (PK), admission_id (FK), nurse_id (FK), recorded_at, temperature, blood_pressure_sys, blood_pressure_dia, pulse, oxygen_saturation, weight, notes
medicines	medicine_id (PK), generic_name, brand_name, category, unit, description
medicine_inventory	inventory_id (PK), medicine_id (FK, UNIQUE), quantity_available, reorder_level, last_updated
prescriptions	prescription_id (PK), admission_id (FK), doctor_id (FK), prescribed_at, notes
prescription_items	item_id (PK), prescription_id (FK), medicine_id (FK), dosage, frequency, duration_days, instructions
media_files	file_id (PK), entity_type, entity_id, file_name, file_type, file_data (BYTEA), firebase_url, uploaded_at, uploaded_by (FK→users)
lab_orders	order_id (PK), admission_id (FK), doctor_id (FK), test_name, ordered_at, status, priority
lab_results	result_id (PK), order_id (FK, UNIQUE), tech_id (FK), result_summary, result_file_id (FK→media_files), resulted_at, notes

continued on next page...

tabheadblue	
-------------	--

bills	bill_id (PK), admission_id (FK), generated_by (FK→users), generated_at, total_amount, discount, paid_amount, status
softgray bill_items	bill_item_id (PK), bill_id (FK), service_type, description, quantity, unit_price, total_price (<i>GENERATED: quantity × unit_price</i>)
payments	payment_id (PK), bill_id (FK), paid_at, amount, payment_method, received_by (FK→users)
softgray alerts	alert_id (PK), alert_type, entity_id, message, is_resolved, created_at
audit_log	log_id (PK), user_id (FK→users), action, table_name, record_id, timestamp, old_value (JSONB), new_value (JSONB)

7.2 Backend Database

- PostgreSQL** — primary relational database (all 21 tables, stored procedures, functions, views, triggers).
- Firebase Firestore** — real-time bed status synchronisation and cloud backup.

7.3 Frontend Technology

- React + Vite** — component-based SPA with role-gated sidebar navigation.

7.4 Use of Advanced Database Features

a) PostgreSQL Functions

- calculate_age(dob DATE)** — Returns a patient's current age; used inside `active_admissions_view` so age is always live without storing it.
- calculate_bill_total(admission_id)** — Rolls up `bill_items.total_price` into the bill header; called by the `generate_bill` and `discharge_patient` procedures as the single source of truth.
- bed_occupancy_rate(ward_id)** — Computes percentage occupancy per ward; reused by `ward_occupancy_view` to avoid duplicate window arithmetic.
- average_vital(admission_id, vital_name)** — Computes the average of any vital sign over an admission using dynamic SQL with a whitelist guard against injection; powers the Vitals modal averages endpoint.

b) Stored Procedures

- admit_patient(patient_id, bed_id, doctor_id, type)** — Transactional admit: validates bed availability, inserts admission, marks bed occupied, and writes an audit entry all-or-nothing — prevents double-bookings.
- discharge_patient(admission_id, discharged_by)** — Transactional discharge: updates status, frees the bed, recomputes bill total via `calculate_bill_total`, and audits — eliminates partial-state bugs.

3. **generate_bill(admission_id, generated_by)** — Computes bed-days, lab, and prescription costs in one call, creates the bill header and itemised lines, prevents duplicate bills, and audits — billing logic stays entirely in the database.

c) Views

1. **active_admissions_view** — Joins admissions, patients, doctors, users, beds, and wards in one query and exposes `patient_age`; powers the Admissions page without client-side joins.
2. **doctor_schedule_today** — Counts total, pending, completed, and cancelled appointments per doctor for `CURRENT_DATE` using `FILTER` clauses; drives the per-doctor schedule widget.
3. **pending_lab_orders_view** — Shows pending/in-progress orders sorted `STAT` → `urgent` → `routine`; powers the Lab Technician queue.
4. **ward_occupancy_view** — Per-ward bed counts by status plus `occupancy_rate_pct`; powers the Beds page hero section.
5. **top_medicines_this_month** — Top prescribed medicines this calendar month with current stock alongside; helps pharmacists spot popular-but-low SKUs.

d) Triggers

1. **trg_bed_on_assign** (AFTER INSERT/UPDATE on admissions) — Auto-marks a bed as occupied the moment an admission is created or transferred.
2. **trg_bed_on_discharge** (AFTER UPDATE OF status on admissions) — Automatically frees a bed to available when a patient is discharged.
3. **trg_decrement_inventory** (AFTER INSERT on prescription_items) — Every prescribed medicine line item auto-decrements `medicine_inventory.quantity_available`.
4. **trg_low_stock_alert** (AFTER UPDATE on medicine_inventory) — Inserts a `low_stock` alert into `alerts` whenever stock drops below `reorder_level`; surfaced in the TopBar notification bell.
5. **trg_admission_audit** (AFTER INSERT/UPDATE/DELETE on admissions) — Writes a full JSONB before/after snapshot to `audit_log` for every admission change.
6. **trg_abnormal_vitals** (AFTER INSERT on vitals) — Checks temperature, `SpO2`, and pulse against safe ranges and inserts an `abnormal_vital` alert automatically — no manual escalation needed.

e) Indexes

1. **idx_patients_cnic, idx_patients_name** — Keeps the global patient search (by name and CNIC) fast under heavy load.
2. **idx_beds_status** — Makes ward occupancy count queries cheap on the Beds page; 14 indexes total cover all hot lookup columns across `admissions`, `appointments`, `lab_orders`, `medicines`, `bills`, `alerts`, `audit_log`, `vitals`, `prescriptions`, and `users`.